COMP 418/518: Homework 2

[Weight of homework: 25% of the final grade]

Dev Sanghvi (ds221), Ansh Dabral (ad258)

Due on February 25, 2026
(released on February 6, 2026)

Instructions: For this homework, you should work in groups of 3–4 people (i.e., the groups that have been declared and accepted). Only one member of the group should submit on Gradescope. All group members must be included in the submission on the Gradescope system (otherwise no credit will be given). Also specify in your submission file all the members of your group (name and NetID) using the `\authors` macro. The submission must be typed in LaTeX using the provided template. The use of AI systems (including, but not limited to, ChatGPT, Gemini, Copilot, and Claude) is entirely prohibited. No discussion or collaboration is allowed outside of the group. Late submissions are not accepted.

You should provide *pseudocode* for all the algorithms that you describe.

Note: We will request that you upload the LaTeX source for your submission. More information regarding how to make the source submission will be made available in a few days.

Grading: The homework will be graded based on correctness, completeness, and the quality of the provided solutions.

Problem 1 [20 points]

Many IoT applications involve the processing of signals that are generated by sensors. Processing these signals often involves *filtering*, a signal processing technique used to remove noise from the signal. The moving (sliding) average that we have seen in class is an example of an *FIR (Finite Impulse Response)* filter that removes high-frequency noise.

You are asked to design a streaming algorithm for another FIR filter, which is a sliding-window computation. The size of the window is W. Assume that the input stream consists of real numbers. At every step (after the first W stream elements have been received), the filter should output the quantity

$$x_1 + 2x_2 + 3x_3 + \cdots + Wx_W,$$

where $x_1, \ldots, x_W$ are the last W elements of the stream (i.e., the current window with x_W being the most recent element).

Describe the most space- and time-efficient streaming algorithm that you can think of. Explain why your algorithm is correct. Give a precise description of the resources (space and time per item) that your algorithm uses.

Solution

Algorithm Description:
We will maintain a circular buffer to store the elements of the current window.

Variables being maintained:

1. S: The simple sum of the elements in the current window.
2. O: The weighted sum of the elements in the current window (your target output).

Algorithm 1 Streaming FIR Filter (W-size window)

Initialization:
Read the first W elements into an array *buffer* of size W.
$S \leftarrow \sum_{i=1}^{W} \mathit{buffer}[i-1]$
$O \leftarrow \sum_{i=1}^{W} i \cdot \mathit{buffer}[i-1]$
$\mathit{idx} \leftarrow 0$ ▷ Pointer to the oldest element
Output O
Streaming Update (for each new element x_{new}):
$x_{\text{old}} \leftarrow \mathit{buffer}[\mathit{idx}]$
$O \leftarrow O - S + W \cdot x_{\text{new}}$
$S \leftarrow S - x_{\text{old}} + x_{\text{new}}$
$\mathit{buffer}[\mathit{idx}] \leftarrow x_{\text{new}}$ ▷ Overwrite oldest element
$\mathit{idx} \leftarrow (\mathit{idx} + 1) \bmod W$ ▷ Update index circularly
Output O

Proof of Correctness: To prove the algorithm is correct, we must show that our update rule produces the correct new weighted sum algebraically.
Let the previous window consist of elements $x_1, x_2, \ldots, x_W$, where x_1 is the oldest and x_W is the most recent. Our running variables represent the following:

$$S = x_1 + x_2 + x_3 + \cdots + x_W$$
$$O = 1x_1 + 2x_2 + 3x_3 + \cdots + Wx_W$$

When a new element x_{new} arrives, the new window becomes $x_2, x_3, \ldots, x_W, x_{\text{new}}$. The correct new weighted sum O_{new} that the filter should output is:

$$O_{\text{new}} = 1x_2 + 2x_3 + \cdots + (W-1)x_W + Wx_{\text{new}}$$

Let's evaluate the algebraic expression used in our streaming algorithm, $O - S$:

$$O - S = (1x_1 + 2x_2 + 3x_3 + \cdots + Wx_W) - (x_1 + x_2 + x_3 + \cdots + x_W)$$

Subtracting term by term, the x_1 cancels out completely, and the coefficients for the rest of the terms decrease by exactly 1:

$$O - S = 1x_2 + 2x_3 + \cdots + (W-1)x_W$$

Notice that $O-S$ neatly handles the shifted weights for all the surviving elements from the previous window. To get the final O_{new}, we just need to add the new element multiplied by its maximum weight, W:

$$(O - S) + Wx_{\text{new}} = 1x_2 + 2x_3 + \cdots + (W-1)x_W + Wx_{\text{new}}$$

This precisely matches O_{new}. The simple sum is also updated correctly:

$$S_{\text{new}} = S - x_1 + x_{\text{new}} = x_2 + x_3 + \cdots + x_W + x_{\text{new}}$$

Because the simple sum S is always correctly maintained, the weighted sum O will continue to be correctly computed in all subsequent streaming steps.

Resources:
Time per item: $O(1)$. After the initial window is filled, processing every subsequent item from the stream requires a strictly fixed number of operations: two additions/subtractions, one multiplication, one array read, one array write, and one modulo operation. This computational cost is entirely independent of the window size W, making it optimally efficient for real-time IoT streaming.
Space: $O(W)$. We must maintain a circular buffer of size W to store the past elements so we can accurately retrieve and subtract the oldest element (x_{old}) from our running sum. We also use a few scalar variables (S, O, idx), which take $\mathcal{O}(1)$ space. The overall space complexity is dominated by the buffer, resulting in $\mathcal{O}(W)$ space. This is the absolute minimum space required; without storing the actual elements of the window, you cannot exactly identify and subtract the exiting element's contribution.

Problem 2 [30 points]

Suppose that the input stream is a sequence of measurements from a smart IoT sensor. The measurements are integers, i.e., elements of $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$. We use the notation $x[n]$, where $n \in \mathbb{N}$, for the input stream. We also call x the *input signal*. The value $x[n]$ is the sensor measurement at time instant n.

We want to perform a sliding-window computation, where $W \geq 2$ is the size of the window. The window at a time instant $n \in \mathbb{N}$ contains the measurements

$$x[\max(0, n - W + 1)], \ldots, x[n - 1], x[n] \ .$$

For example, if $W = 3$, then the window at time instant $n = 5$ contains the measurements $x[3], x[4], x[5]$. We use the notation $y[n]$, where $n \in \mathbb{N}$, for the output stream we want to compute, i.e., the *output signal*. The value $y[n]$ is the output at time instant n. Let $T_1, T_2 \in \mathbb{Z}$ be two *threshold* values that satisfy $T_1 < T_2$. Define the output value

$$y[n] = \begin{cases} 1, & x[i] \in [T_1, T_2) \text{ and } x[j] \in [T_2, \infty) \text{ for some } i, j \in \{\max(0, n - W + 1), \ldots, n\} \text{ with } i < j \\ 0, & \text{otherwise} \end{cases}$$

for every time instant $n \in \mathbb{N}$. In other words, $y[n] = 1$ iff the window (at time instant n) contains a measurement in the interval $[T_1, T_2) = \{v \mid T_1 \leq v < T_2\}$ followed by a measurement in the interval $[T_2, \infty) = \{v \mid v \geq T_2\}$.

Describe the most space- and time-efficient streaming algorithm that you can think of for this sliding-window computation. Explain why your algorithm is correct. Give a precise description of the resources (space and time per item) that your algorithm uses.

Solution

Algorithm Description

The problem asks us to output $y[n] = 1$ if the current window of size W contains a measurement in $[T_1, T_2)$ followed by a measurement in $[T_2, \infty)$.
Instead of maintaining a buffer of the last W measurements, we only need to keep track of the *most recent* indices that satisfy these conditions. Let (i, j) be a pair of indices such that $i < j$, $x[i] \in [T_1, T_2)$, and $x[j] \geq T_2$. We call this a "valid pair". For $y[n]$ to be 1, there must exist a valid pair within the current window.

We maintain two state variables:

1. idx_{T1}: The index of the most recently seen measurement in $[T_1, T_2)$.
2. $idx_{success}$: The index i of the most recent valid pair (i, j).

Algorithm 2 Streaming IoT Threshold Detection

Initialization:
$idx_{T1} \leftarrow -1$ ▷ Tracks the latest index i where $x[i] \in [T_1, T_2)$
$idx_{success} \leftarrow -1$ ▷ Tracks the latest i that forms a valid pair
$n \leftarrow 0$ ▷ Current time instant
Streaming Update (for each new measurement $x[n]$):
if $T_1 \leq x[n] < T_2$ **then**
    $idx_{T1} \leftarrow n$
else if $x[n] \geq T_2$ **then**
    $idx_{success} \leftarrow idx_{T1}$
end if
if $idx_{success} \geq \max(0, n - W + 1)$ **then**
    $y[n] \leftarrow 1$
else
    $y[n] \leftarrow 0$
end if
Output $y[n]$
$n \leftarrow n + 1$

Correctness

To prove correctness, we must show that $y[n] = 1$ if and only if a valid pair (i, j) exists within the window $[\max(0, n - W + 1), n]$.

1. For a valid pair (i, j) to be in the current window, we must have $\max(0, n - W + 1) \leq i < j \leq n$. Notice that because $j > i$, if i is inside the window bound ($i \geq n - W + 1$), j is guaranteed to be inside the window bound as well (since j arrived at or before step n). Therefore, the survival of the pair in the sliding window depends *entirely* on whether index i has fallen out of the window.

2. Suppose there are multiple valid pairs in the stream. The pair that will survive the longest in the sliding window is the one with the maximum starting index i.

Our algorithm tracks the most recent $x[k] \in [T_1, T_2)$ using idx_{T1}. Thus, idx_{T1} is always the largest possible starting index available.

Whenever a valid closing element $x[n] \geq T_2$ arrives, it forms a pair with the most recently seen starting element. We update $idx_{success} \leftarrow idx_{T1}$. This locks in the largest known index i that is part of a valid pair.

3. Finally, the algorithm outputs 1 if $idx_{success} \geq \max(0, n - W + 1)$. This correctly checks if the best possible starting index i is still within the current bounds of the sliding window. If the best i has expired (fallen out), then all older starting indices have also expired, meaning no valid pairs remain, and the algorithm correctly outputs 0.

Resources

Time per item: $\mathcal{O}(\mathbf{1})$. Processing a new item requires at most two comparisons to check the thresholds, one assignment to update the state variables, one subtraction to calculate the window bound, and one comparison to determine $y[n]$. This is strictly constant time per measurement.

Space: $\mathcal{O}(\mathbf{1})$. Unlike standard sliding window algorithms that require $\mathcal{O}(W)$ space to maintain a buffer, this algorithm does not need to remember the measurements themselves. We only store three scalar integers: idx_{T1}, $idx_{success}$, and n. Thus, the space complexity is optimally $\mathcal{O}(1)$ regardless of the window size W.

Problem 3 [15 points]

Recall from the lectures the "sliding max" problem, that is, the streaming problem of calculating the maximum over a sliding window.

Give an algorithm for the sliding max problem with worst-case time per item $O(\log W)$, where W is the size of the window. Explain why your algorithm is correct. What is the space and total time complexity of your algorithm?

Solution

Algorithm Description

While a deque-based approach provides $\mathcal{O}(1)$ amortized time, its worst-case time per item can be $\mathcal{O}(W)$. To strictly bound the worst-case time per item to $\mathcal{O}(\log W)$, we can use a **Balanced Binary Search Tree (BBST)** paired with a **circular buffer**.
The circular buffer keeps track of the chronological arrival of the items so we know exactly which item is expiring and needs to be removed. The BBST maintains the sorted order of the elements currently in the window, allowing us to query the maximum efficiently.

We maintain the following structures:
Tree: A BBST that stores the elements of the current window.
Buffer: An array of size W that acts as a circular queue.
idx: A counter modulo W to track the oldest element in the buffer.

Algorithm 3 Sliding Max with $\mathcal{O}(\log W)$ worst-case time

```
Initialization:
Initialize an empty BBST Tree.
Initialize an array Buffer of size W.
idx ← 0
n ← 0                                          ▷ Number of elements processed
Streaming Update (for each new element x[n]):
if n ≥ W then
    x_old ← Buffer[idx]
    Tree.removeOne(x_old)                      ▷ Remove one instance of the expired element
end if
Tree.insert(x[n])                              ▷ Insert the new element
Buffer[idx] ← x[n]                             ▷ Overwrite the oldest element in the buffer
idx ← (idx + 1) mod W                          ▷ Update circular pointer
if n ≥ W − 1 then
    current_max ← Tree.findMax()               ▷ The largest element in the BBST
    Output current_max
end if
n ← n + 1
```

Correctness

The algorithm correctly computes the sliding window maximum because the data structures perfectly mirror the current state of the window at any time step n.
Synchronization: The circular buffer continuously holds exactly the last W elements from the stream. When the window slides, the exact element that falls out of the W-sized window is identified by the buffer.
Tree State: By explicitly inserting the newest element $x[n]$ and strictly removing the exactly matched oldest element x_{old} from the BBST, the tree is guaranteed to contain the exact same multiset of W elements as the current sliding window.
Max Query: Because a BBST maintains its elements in sorted order, querying the maximum element (*Tree.findMax()*) is guaranteed to return the correct maximum value among the current W items.

Complexity Analysis

Worst-case time per item: $\mathcal{O}(\log \mathbf{W})$. For each incoming item, we perform at most one insertion and one deletion in the BBST. Because the BBST never contains more than W elements, both *insert* and *remove* operations take strictly $\mathcal{O}(\log W)$ time in the worst case. Finding the maximum takes $\mathcal{O}(\log W)$ time (or $\mathcal{O}(1)$ if a pointer to the rightmost node is maintained). Array operations take $\mathcal{O}(1)$ time. Thus, the total worst-case time per item is bounded by $\mathcal{O}(\log W)$.

Space Complexity: $\mathcal{O}(\mathbf{W})$. The circular buffer stores exactly W elements. The BBST also stores at most W elements (with standard $\mathcal{O}(1)$ overhead per node for pointers and balance factors). Therefore, the overall space complexity is strictly $\mathcal{O}(W)$.

Total Time Complexity: $\mathcal{O}(\mathbf{N} \log \mathbf{W})$. If the total length of the input stream is N items, and we process each item in $\mathcal{O}(\log W)$ worst-case time, the total time to process the entire stream is $\mathcal{O}(N \log W)$.

Problem 4 [40 points]

Suppose that the input items are drawn from the set $\{0, 1, \ldots, R-1\}$, where R is a (small) *constant*.

(1) Give a space-optimal (asymptotically) algorithm for the sliding max problem (the window size is W). Measure space (memory) in bits.

(2) Prove a space lower bound (for the problem) that matches the upper bound for your algorithm.

Solution

(1): Space-Optimal Algorithm

Since the input values are restricted to a small constant set $\{0, 1, \ldots, R-1\}$, we do not need to store the values themselves. Instead, we can simply keep track of the *age* of the most recent occurrence of each possible value. If a value's age is less than W, it is currently inside the window.

We maintain an array D of size R, where $D[v]$ stores the number of time steps since the value v was last seen. If v has not been seen in the last W steps, we cap $D[v]$ at W.

Algorithm 4 Space-Optimal Constant-Alphabet Sliding Max

```
Initialization:
Initialize an array D[0 ... R − 1] such that D[v] = W for all v.
n ← 0
Streaming Update (for each new element x[n]):
for v ← 0 to R − 1 do
    if D[v] < W then
        D[v] ← D[v] + 1                          ▷ Increment the age of all active values
    end if
end for
D[x[n]] ← 0                                      ▷ The new element has an age of 0
if n ≥ W − 1 then
    max_val ← 0
    for v ← R − 1 down to 0 do                   ▷ Check from largest possible value down
        if D[v] < W then
            max_val ← v
            break
        end if
    end for
    Output max_val
end if
n ← n + 1
```

Space Complexity

The algorithm maintains exactly R integers. Because we cap the age at W, each integer $D[v] \in \{0, 1, \ldots, W\}$. Storing a number up to W requires $\lceil \log_2(W+1) \rceil$ bits. Therefore, the total space required is:

$$S(W) = R \cdot \lceil \log_2(W+1) \rceil \text{ bits}$$

Since R is a constant, the asymptotic space complexity is strictly $\mathcal{O}(\log \mathbf{W})$ **bits**.

(2): Space Lower Bound Proof

To prove a matching lower bound of $\Omega(\log W)$ bits, we must show that any deterministic streaming algorithm requires at least this much memory to correctly answer sliding max queries. We will use a state-distinguishability argument.

Suppose the algorithm has processed a prefix of the stream. Its internal memory must encode enough information to answer all future queries correctly. Let us construct a family of adversarial window states of length W that all require different memory states.

Consider strings of length W over the alphabet $\{0, \ldots, R-1\}$ that contain exactly one instance of each value $v \in \{1, 2, \ldots, R-1\}$, and 0 everywhere else. Let $p_v \in \{1, 2, \ldots, W\}$ denote the index (position) of value v inside this window. We restrict our family of strings to those where the non-zero values appear in strictly decreasing order from left to right. That is, their positions satisfy:

$$1 \leq p_{R-1} < p_{R-2} < \cdots < p_1 \leq W$$

Suppose the algorithm has just processed one of these strings. Now, an adversary feeds a continuous stream of 0s.

- Initially, the maximum is $R-1$.
- When exactly $W - p_{R-1}$ zeros have been appended, the value $R-1$ falls out of the sliding window. Because $p_{R-2} > p_{R-1}$, the value $R-2$ is still in the window. Thus, the output transitions exactly from $R-1$ to $R-2$.
- Generally, the output transitions from k to $k-1$ exactly when $W - p_k$ zeros have been appended.

Because the exact number of zeros required to cause a drop in the maximum uniquely identifies the position p_k for every $k \in \{1, \ldots, R-1\}$, **any two strings with a different set of positions must produce a different sequence of future outputs.** Therefore, the algorithm must be in a strictly different internal memory state for every valid combination of positions.

How many such combinations exist? We are choosing $R-1$ distinct positions out of W possible slots. The number of valid states is exactly:

$$\binom{W}{R-1}$$

By information theory, to uniquely represent at least $\binom{W}{R-1}$ different states, the algorithm requires M bits of memory, where:

$$M \geq \log_2 \binom{W}{R-1}$$

Using the definition of the binomial coefficient:

$$\log_2 \binom{W}{R-1} = \log_2 \left(\frac{W(W-1)\ldots(W-R+2)}{(R-1)!} \right)$$

For large W and constant R, this is dominated by the W^{R-1} term:

$$M \geq (R-1)\log_2 W - \log_2((R-1)!)$$

Because R is a constant, $(R-1)$ is a constant factor and $\log_2((R-1)!)$ is a constant term. Therefore, the memory space required is:

$$M = \mathbf{\Omega}(\log \mathbf{W}) \text{ bits}$$

This lower bound matches the $\mathcal{O}(\log W)$ bit upper bound of our algorithm, proving that the algorithm is asymptotically space-optimal.

Problem 5 [20 points]

An ***IoT device A*** generates *messages* that are nested lists/sets of (arbitrarily large) natural numbers. We will define now the set D, which should be thought of as an abstract mathematical representation of such messages. Formally, D is the smallest set that satisfies the following properties (which are implicitly universally quantified):

1. [Number] If $n \in \mathbb{N}$ then $n \in D$ ("every natural number is a message").
2. [List] If $x_1, x_2, \ldots, x_n \in D$ then $[x_1, x_2, \ldots, x_n] \in D$.
3. [Set] If $x_1, x_2, \ldots, x_n \in D$ then $\{x_1, x_2, \ldots, x_n\} \in D$.

Empty lists $[\,]$ and empty sets $\{\}$ are messages. For example, $[4, \{2, 3\}, [1, 2, \{\}], [6, [\,]], 7]$ is a message. The messages are transmitted in serialized form, where numbers are represented as sequences of decimal digits. A *serialized message* is a sequence over the following alphabet Σ of symbols:

`0 1 2 3 4 5 6 7 8 9 [ ] { } ,`

It is also allowed to have (any number of) space symbols separating numbers, commas, brackets, and braces. For example, this is a serialized message (observe the presence of space symbols):

`[4, {2, 3}, [1, 2, {}], [6, []], 7]`

The ***IoT device B*** receives a stream over Σ (***message stream***), which should be a sequence of serialized messages that are separated by (any number of) space symbols. For example,

`[ ] 100 {1} [2, 3] [4, {7,8}, 5] { }`

is a stream that consists of 6 complete serialized messages.

Describe a streaming algorithm for Device B that processes an input message stream. As soon as the algorithm sees a complete (serialized) message, it should output the total number of messages processed so far. This means that the algorithm should not produce output at every step. As soon as the algorithm notices that a message is *malformed* (e.g., `[1, 2}` or `[1, [2, 3]]]` or `[1, 2,]`), it should start emitting the error value -1 at every step.

Explain why your algorithm is correct. Give a precise description of the resources (space and time per item) that your algorithm uses.

Solution

Algorithm Overview

We use a **finite-state automaton** combined with a **stack** S to track bracket/brace nesting. The automaton has five normal states plus an error state:

- `TOP`: Between top-level messages. Expecting the start of a new message or a space.
- `NUM`: Reading digits of a number.
- `OPEN`: Just opened a bracket/brace. Expecting an element or a matching close.
- `SEP`: Just saw a comma. Expecting the next element.
- `ELEM`: Just finished an element inside a list/set. Expecting a comma or a matching close.
- `ERR`: Malformed input detected. Emit -1 at every subsequent step.

We also maintain a counter C (number of complete messages).

Pseudocode

Algorithm 5 Streaming Message Parser (Part 1)

```
Initialization:
state ← TOP                                          ▷ Current parser state
S ← empty stack                                      ▷ Tracks bracket/brace nesting
C ← 0                                                ▷ Number of complete messages
Streaming Update (for each new symbol σ ∈ Σ):
if state = ERR then
    Output −1
    return
end if
emit_C ← false                                       ▷ Flag to buffer output until step end
if state = NUM ∧ S is empty ∧ σ is not a digit then
    C ← C + 1
    emit_C ← true
    state ← TOP
end if
if σ is a space then
    if state = NUM ∧ S is non-empty then
        state ← ELEM
    end if
end if
if σ is a digit then
    if state ∈ {TOP, OPEN, SEP} then
        state ← NUM
    else if state = ELEM then
        state ← ERR
    end if
end if
if σ ∈ {[, {} (open bracket/brace) then
    if state ∈ {TOP, OPEN, SEP} then
        push σ onto S
        state ← OPEN
    else
        state ← ERR
    end if
end if
```

Algorithm 5 Streaming Message Parser (Part 2)

```
if σ ∈ {], }} (close bracket/brace) then
    if state = SEP then
        state ← ERR
    else if state ∈ {NUM, OPEN, ELEM} ∧ S is non-empty then
        o ← S.top()
        if (o = [ ∧ σ = ]) ∨ (o = { ∧ σ = }) then
            pop S
            if S is empty then
                C ← C + 1
                state ← TOP
                emit_C ← true
            else
                state ← ELEM
            end if
        else
            state ← ERR
        end if
    else
        state ← ERR
    end if
end if
if σ = , then
    if state ∈ {NUM, ELEM} ∧ S is non-empty then
        state ← SEP
    else
        state ← ERR
    end if
end if
if state = ERR then
    Output −1                                   ▷ Prioritize error reporting
else if emit_C then
    Output C
end if
```

Correctness

The automaton enforces the grammar of valid messages:

- A message is either a number (sequence of digits), a list `[...]`, or a set {...}.
- List/set elements are separated by commas. Empty lists `[]` and sets { } are allowed (via the `OPEN` → close transition).
- The stack S ensures correct nesting and matching of brackets/braces.
- Transitions to `ERR` capture all malformed cases: mismatched brackets (`[1,2}`), extra closing brackets (`[1,[2,3]]]`), trailing commas (`[1,2,]`), missing commas, commas at top level, and double commas.

- Upon detecting a malformed condition, the algorithm transitions to `ERR` and **immediately** outputs -1 on the exact same step, as well as at every subsequent step.
- Top-level numbers do not require a trailing space to be considered complete; the algorithm detects the end of a top-level number natively upon encountering any non-digit character (like '[' or '{') or via the absence of items on the stack.

Example trace. For the stream "`[ ] 100 {1}`", the algorithm transitions: `TOP` $\xrightarrow{[}$ `OPEN` $\xrightarrow{\sqcup}$ `OPEN` $\xrightarrow{]}$ `TOP` (output $C=1$) $\xrightarrow{\sqcup}$ `TOP` $\xrightarrow{1}$ `NUM` $\xrightarrow{0}$ `NUM` $\xrightarrow{0}$ `NUM` $\xrightarrow{\sqcup}$ `TOP` (output $C=2$) $\xrightarrow{\{}$ `OPEN` $\xrightarrow{1}$ `NUM` $\xrightarrow{\}}$ `TOP` (output $C=3$).

Resource Analysis

- **Time per item:** $O(1)$. Each symbol requires a constant number of operations (check state, possibly push/pop the stack, update counter).
- **Space:** $O(d_{\max} + \log n)$ bits, where $d_{\max}$ is the maximum nesting depth and n is the stream length. The stack uses $d_{\max}$ bits (1 bit per level, distinguishing `[` from {). The counter C uses $O(\log n)$ bits. The automaton state uses $O(1)$ bits (one of 6 states = 3 bits). In the worst case, $d_{\max} = O(n)$ (e.g., `[[[...]]]`), so the space is $O(n)$ bits.

Problem 6 [125 points]

Implement the approximation algorithm for counting 1's over a sliding window using the provided code files. Include the benchmarking results and any other relevant information below.

- Zip up the entire folder with your solution (only the source code!) and submit it on Gradescope.
- Your implementation should only call malloc ***once*** (during the initialization of the state). After this call of malloc during initialization, there should be no other dynamic memory allocation. The purpose of this restriction is to be faithful to an embedded IoT device that has a small amount of memory and offers no dynamic memory allocation.
- All the data structures used by your algorithm should be implemented from scratch.

Solution

Algorithm Overview

We implement the DGIM (Datar–Gionis–Indyk–Motwani) approximation algorithm for counting 1's over a sliding window of size W. The algorithm maintains a collection of *buckets*, where each bucket stores:

- A **timestamp** (the time of the most recent 1 it represents), and
- A **power** p (the bucket represents 2^p ones).

Setting the parameter k determines the accuracy: we allow at most $r+1 = k+1$ buckets per power level.

Data Structure

We use **per-power-level ring buffers** (circular queues), each storing only timestamps for buckets of that power. Specifically:

- For each power level $p \in \{0, 1, \ldots, \lfloor \log_2 W \rfloor + 2\}$, we maintain a separate ring buffer of capacity $r + 2$.
- All ring buffers are allocated from a **single contiguous block** via one call to `malloc`. The total memory is $(r + 2) \times$ (number of power levels) $\times$ sizeof(`uint32_t`) bytes.
- We also maintain a running `total_sum` of all bucket sizes and a `highest_power` tracker.

This design achieves $O(1)$ amortized time per merge (just pop from one ring and push to another), and avoids any additional dynamic memory allocation after initialization.

Processing Each Item

On each incoming bit:

1. **Expire**: Remove the oldest bucket (at the tail of the highest occupied power level) if its timestamp is outside the window.
2. **Insert**: If the bit is 1, push a new entry (timestamp, power 0) onto ring 0.

3. **Merge cascade**: Starting from power $p = 0$, if the ring for power p has $r+2$ entries, pop the two oldest, and push one merged entry (with the more recent of the two timestamps) onto the ring for power $p+1$. Repeat for $p+1$ if needed.

4. **Query**: Return $\texttt{total_sum} - 2^{p_{\max}} + 1$, where $p_{\max}$ is the power of the oldest bucket.

Estimate and Error Analysis

The algorithm's estimate for the number of 1's is:

$$\hat{C} = \texttt{total_sum} - 2^{p_{\text{oldest}}} + 1$$

where p_{oldest} is the power (size $2^{p_{\text{oldest}}}$) of the oldest (highest-power) bucket currently in the window.

No overestimation: The oldest bucket has not expired, meaning its timestamp (which corresponds to the most recent 1 it represents) is strictly inside the window. Therefore, at least 1 of its $2^{p_{\text{oldest}}}$ constituent 1's is guaranteed to be inside the window. Thus, the true count C must be at least:

$$C \geq \texttt{total_sum} - 2^{p_{\text{oldest}}} + 1 = \hat{C}$$

Rigorous Error Bound Proof: The absolute error is bounded by the unknown remaining 1's tightly packed in the oldest bucket, which is at most $2^{p_{\text{oldest}}} - 1$. To bound the relative error $\epsilon = \frac{C-\hat{C}}{C}$, we must mathematically prove a lower bound on the true count C.

By the construction of the DGIM merging rules, we only merge two buckets of size 2^{p-1} into a bucket of size 2^p when we encounter a total of $r+2$ buckets of size 2^{p-1}. After this merge, exactly r buckets of size 2^{p-1} remain. Because we set $r = k$, this implies that the existence of **any** bucket of size 2^p strictly guarantees the existence of *at least* k buckets of size 2^{p-1}, k buckets of size 2^{p-2}, and so on down to k buckets of size 2^0.

Therefore, if the oldest bucket has power $p_{\text{oldest}} = p$, the `total_sum` of all buckets currently maintained by the algorithm is bounded below by the oldest bucket itself plus the guaranteed k buckets at every smaller power level:

$$\begin{aligned}\texttt{total_sum} &\geq 2^p + \sum_{i=0}^{p-1} k \cdot 2^i \\ &= 2^p + k(2^p - 1)\end{aligned}$$

Since the true count $C \geq \hat{C} = \texttt{total_sum} - 2^p + 1$, we substitute our lower bound for `total_sum`:

$$C \geq [2^p + k(2^p - 1)] - 2^p + 1 = k(2^p - 1) + 1$$

The maximum possible absolute error is $2^p - 1$. Thus, the maximum relative error is strictly bounded by:

$$\text{Relative Error} = \frac{\text{error}}{C} \leq \frac{2^p - 1}{k(2^p - 1) + 1} < \frac{2^p - 1}{k(2^p - 1)} = \frac{1}{k}$$

By choosing $k \geq 1/\epsilon$, the relative error is guaranteed to be strictly less than ϵ, satisfying the DGIM approximation guarantee. □

Complexity

- **Space**: $O(k \cdot \log W)$ timestamps $= O(k \log W)$ words. A single `malloc` call allocates all memory at initialization.
- **Time per item**: $O(1)$ amortized. Each item causes at most one insertion and one expiry, both $O(1)$. Merges are amortized $O(1)$ since each bucket is merged at most $O(\log W)$ times over its lifetime.

Benchmarking Methodology & Results

Experimental Setup: The algorithms were compiled on an Apple Silicon processor using GCC with full optimizations via the command `gcc -O3 bench.c -o bench -lm`. We benchmarked over an adversarial stream of 50,000,000 items *(all 1's)*. This stream pattern represents the strict, pessimistic worst-case scenario for the DGIM algorithm, as every single bit insertion necessarily triggers a new bucket creation and maximizes the frequency of $O(\log W)$ merge cascades. The window sizes varied across $W \in \{10, \ldots, 1{,}000{,}000\}$ and accuracy parameters $k \in \{10, 100, 1000\}$. Each configuration was run for 3 trials.

Raw Benchmark Data: The table below reports the memory footprint (bytes) and average throughput (Millions of items/sec) spanning all configurations.

	Exact	**Approx** ($k = 10$)		**Approx** ($k = 100$)		**Approx** ($k = 1000$)	
W	Mem	Mem	TP	Mem	TP	Mem	TP
10	10 B	720 B	∼200M	5,040 B	∼176M	48,240 B	∼196M
100	100 B	1,080 B	∼195M	7,560 B	∼196M	72,360 B	∼197M
1,000	1,000 B	1,440 B	∼197M	10,080 B	∼176M	96,480 B	∼202M
10,000	10,000 B	1,920 B	∼184M	13,440 B	∼162M	128,640 B	∼200M
100,000	100,000 B	2,280 B	∼205M	15,960 B	∼182M	152,760 B	∼192M
1,000,000	1,000,000 B	2,640 B	∼192M	18,480 B	∼185M	176,880 B	∼198M

Table 1: Raw benchmarking throughputs (Millions items/sec) and memory bounds (Bytes).

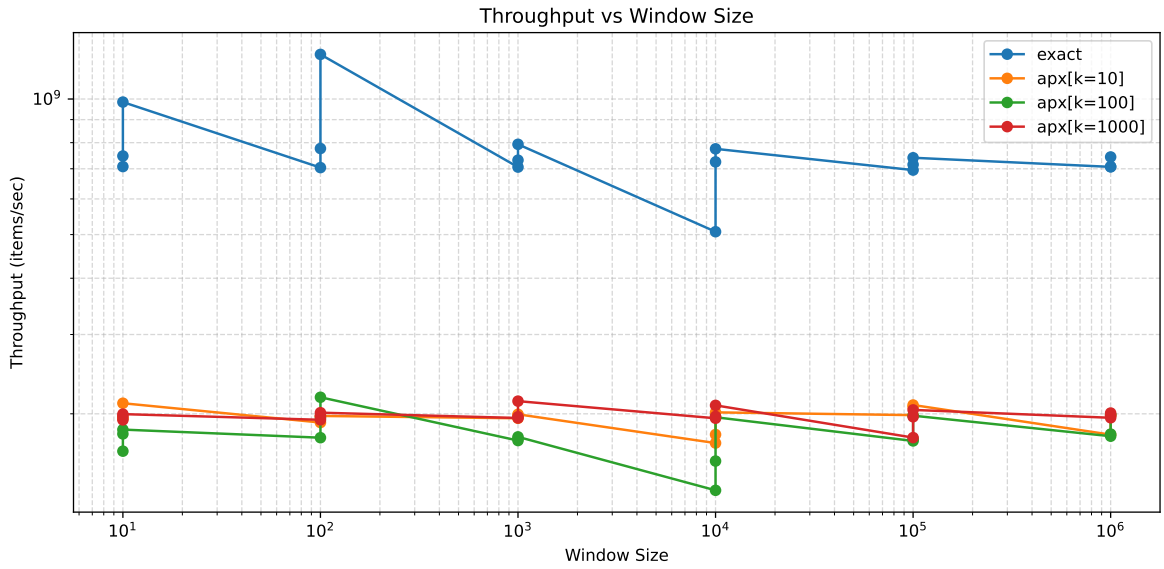

Figure 1: Throughput comparison (items/sec, log scale). The exact algorithm hits ∼760M items/sec on average regardless of W. Despite being stress-tested on its worst-case stream configuration mapping to continuous merge cascades, the approximate algorithm maintains a heavily robust throughput of ∼190M items/sec. Throughput remains relatively insensitive to k and W.

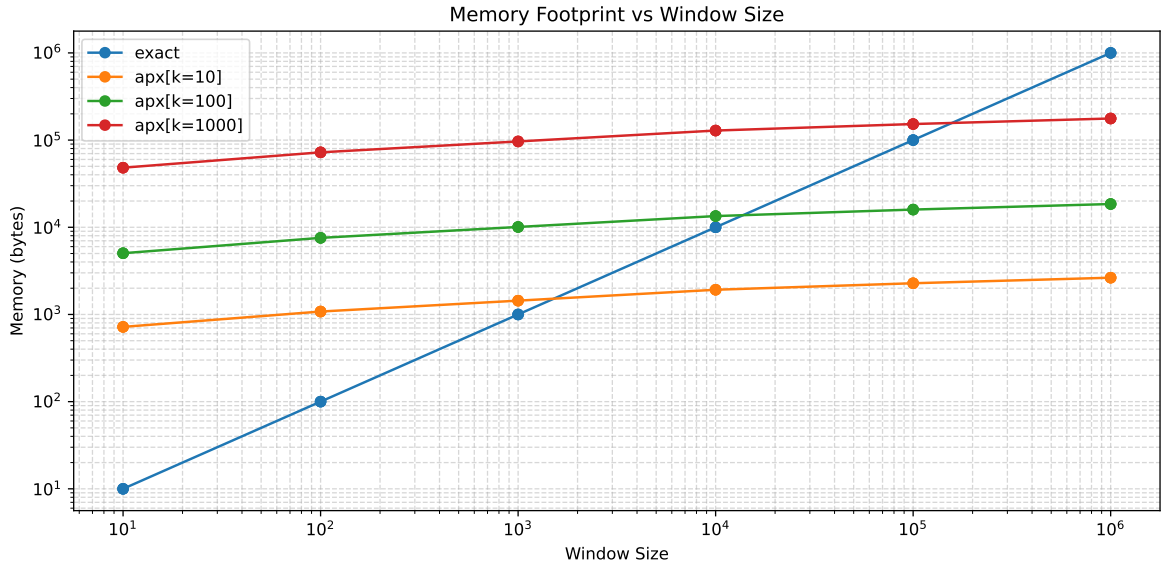

Figure 2: Memory footprint comparison (bytes, log scale). The exact algorithm uses W bytes ($\Theta(W)$). The approximate algorithm uses $O(k \log W)$ bytes — for example, at $W = 1{,}000{,}000$ with $k = 10$, it uses only 2,640 bytes vs. 1,000,000 bytes for exact, a ∼378× reduction.